

Next '26 開發人員主題演講：運用記憶功能強化代理

來源：<https://codelabs.developers.google.com/next26/dev-keynote/enhancing-agents-with-memory?hl=zh-tw>

步驟數：13

瞭解如何使用 Agent Platform 會話管理、Memory Bank 和 AlloyDB RAG，強化 ADK 代理。

目錄

1. [簡介](#)
2. [事前準備](#)
3. [設定環境](#)
4. [使用 Session Management 建立代理](#)
5. [啟用 Memory Bank 的長期學習功能](#)
6. [設定 AlloyDB 以進行 RAG](#)
7. [使用 Spark Lightning Engine 擷取城市規則資料](#)
8. [搭配 AlloyDB 使用 RAG](#)
9. [透過 Agent Skills 取得專家指引](#)
10. [測試代理](#)
11. [部署代理程式](#)
12. [清除所用資源](#)
13. [恭喜](#)

步驟 1 · 約 0 分鐘

1. 簡介

在本程式碼研究室中，您將新增持續性專業知識，進一步提升 ADK 服務專員的效能。您將瞭解如何使用 Agent Platform Sessions 管理對話狀態、透過 Memory Bank 啟用長期學習功能，以及使用 Spark 和 AlloyDB for RAG (檢索增強生成) 整合複雜的城市規則資料。

學習內容

- 設定 **Agent Platform** 工作階段，確保對話持續進行。
- 實作 **Memory Bank**，讓代理程式從先前的互動中學習。
- 使用 **Spark Lightning Engine** 擷取及處理城市規則文件。
- 使用 **AlloyDB** 和向量搜尋功能建構 **RAG** 系統。
- 將強化型代理部署至 Agent Platform。

軟硬體需求

- 網路瀏覽器，例如 [Chrome](#)
- 已[啟用計費功能](#)的 Google Cloud 專案
- 對 Python 和 SQL 有基本瞭解

預估時間：60 分鐘

本程式碼研究室建立的資源費用應低於 \$5 美元。

2. 事前準備

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

建立 Google Cloud 專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，[選取或建立 Google Cloud 專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。

啟動 Cloud Shell

Cloud Shell 是在 Google Cloud 中運作的指令列環境，已預先載入必要工具。

1. 點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。
2. 連至 Cloud Shell 後，請驗證您的驗證：

□□□

```
gcloud auth list
```

3. 確認專案已設定完成：

```
gcloud config get project
```

4. 如果專案未如預期設定，請設定專案：

```
export PROJECT_ID=<YOUR_PROJECT_ID>
gcloud config set project $PROJECT_ID
```

驗證：

```
gcloud auth list
```

確認專案：

```
gcloud config get project
```

視需要設定：

```
export PROJECT_ID=<YOUR_PROJECT_ID>
gcloud config set project $PROJECT_ID
```

啟用 API

執行下列指令，啟用工作階段管理、Spark 處理和 AlloyDB 的所有必要 API：

```
gcloud services enable \  
  aiplatform.googleapis.com \  
  run.googleapis.com \  
  alloydb.googleapis.com \  
  dataproc.googleapis.com \  
  documentai.googleapis.com \  
  storage.googleapis.com \  
  secretmanager.googleapis.com
```

啟用這些 API 後，您就能存取 Agent Platform、受管理 AlloyDB 叢集，以及 Spark 管道的 Dataproc Serverless。

步驟 3 · 約 5 分鐘

3. 設定環境

在本程式碼研究室中，您將使用主題演講存放區中預先設定的環境。

1. 複製存放區並前往專案資料夾：

```
git clone https://github.com/GoogleCloudPlatform/next-26-keynotes
cd next-26-keynotes/devkey/enhancing-agents-with-memory
```

2. 設定 Python 虛擬環境並安裝必要的 ADK 套件：

```
uv venv
source .venv/bin/activate
uv sync
```

設定環境變數

代理程式需要特定設定，才能連線至 Agent Platform 和 AlloyDB。

1. 複製範例環境檔案：

```
cp .env.example .env
```

2. 開啟 `.env` 並更新下列欄位：

- `GOOGLE_CLOUD_PROJECT`：專案 ID。
- `GOOGLE_CLOUD_LOCATION`：`us-central1`。
- `ALLOYDB_CLUSTER_ID`：`rules-db`。

```
GOOGLE_CLOUD_PROJECT=<YOUR_PROJECT_ID>
GOOGLE_CLOUD_LOCATION=global
GOOGLE_GENAI_USE_VERTEXAI=TRUE
GOOGLE_CLOUD_REGION=us-central1
ALLOYDB_CLUSTER_ID=rules-db
```

3. 執行下列輔助指令碼，建立用於對話工作階段和長期記憶的 **Agent Engine 執行個體**。系統會自動在 `.env` 檔案中填入 `AGENT_ENGINE_ID`：

```
uv run utils/setup_agent_engine.py
```

成功後，您應該會看到：

```
Creating Agent Engine instance...  
Successfully created Agent Engine. ID: 1234567890  
Updated .env with AGENT_ENGINE_ID=1234567890
```

步驟 4 · 約 7 分鐘

4. 使用 Session Management 建立代理

在這個步驟中，您將初始化 **Marathon Planner Agent**，這個代理程式可維護多輪對話的對話記錄。這是透過 ADK **App** 類別和 **Agent Platform Sessions** 達成。

初始化 Agent 和 Session 服務

開啟 `planner_agent/agent.py`。您會看到我們如何新增 ADK 類別，整合 **Agent Platform Sessions**。這讓我們能夠隨著時間推移，讓代理程式保持有狀態，並視需要修改內容。

```
from google.adk.agents import LlmAgent
from google.adk.sessions import VertexAiSessionService
from vertexai.agent_engines import AdkApp

PROJECT_ID = os.environ.get("GOOGLE_CLOUD_PROJECT")
REGION = os.environ.get("GOOGLE_CLOUD_REGION", "us-central1")

# Initialize Vertex AI for regional services
if PROJECT_ID:
    vertexai.init(project=PROJECT_ID, location=REGION)

# Define the agent logic
root_agent = LlmAgent(
    name="planner_agent",
    model="gemini-3-flash-preview",
    instruction="You are a helpful marathon planning assistant...",
    tools=[] # We will add tools in the next steps
)

def session_service_builder():
    """Builder for Agent Platform Sessions."""
    return VertexAiSessionService(project=PROJECT_ID, location=REGION)

# Wrap the agent in an AdkApp to manage stateful context
app = AdkApp(
    agent=root_agent,
    session_service_builder=session_service_builder
)
```


5. 啟用 Memory Bank 的長期學習功能

工作階段管理功能會追蹤個別對話，長期記憶體也能做到這點。在這個步驟中，您會將代理連結至 **Agent Platform** 的 **Memory Bank**，這項全代管記憶體服務可供企業使用。

初始化 Memory Bank 服務

Memory Bank 可讓代理在不同工作階段中記住脈絡。更新 `planner_agent/agent.py`，加入記憶體服務：

```
from google.adk.memory import VertexAiMemoryBankService

def memory_service_builder():
    """Builder for Agent Platform Memory Bank."""
    return VertexAiMemoryBankService(
        project=PROJECT_ID,
        location=REGION,
        agent_engine_id=AGENT_ENGINE_ID
    )
```

實作自動記憶體擷取

為確保代理程式從每個回合中學習，我們新增了 `after_agent_callback`。代理程式完成回覆後，系統會觸發這個函式，讓代理程式「消化」工作階段，並將相關記憶儲存至記憶庫。

1. 定義回呼函式：

```
async def auto_save_memories(callback_context):
    """Callback to ingest the session into the memory bank after the turn."""
    # In AdkApp, the memory service is available via the invocation context
    if hasattr(callback_context._invocation_context, 'memory_service') and
        callback_context._invocation_context.memory_service:
        await callback_context._invocation_context.memory_service.add_session_to_memory(
            callback_context._invocation_context.session
        )
```

2. 將回呼附加至 `LlmAgent`：

```
root_agent = LlmAgent(
    # ... other params
    after_agent_callback=[auto_save_memories],
)
```

整合 Agent Platform 工作階段和 Memory Bank 後，代理不再只是無狀態模型，而是有狀態的助理，可維持脈絡並隨著時間從使用者身上學習。

6. 設定 AlloyDB 以進行 RAG

我們需要高效能的資料庫來儲存城市規則資料，才能擷取這類資料。在這個步驟中，您會建立 AlloyDB 叢集，並初始化向量搜尋的資料庫結構定義。

1. 建立 AlloyDB 叢集和主要執行個體

在 Cloud Shell 中執行下列指令，建立叢集及其主要執行個體：

```
# Create the cluster
gcloud alloydb clusters create rules-db \
  --password=postgres \
  --region=us-central1

# Create the primary instance with IAM authentication enabled
gcloud alloydb instances create rules-db-primary \
  --instance-type=PRIMARY \
  --cpu-count=2 \
  --region=us-central1 \
  --cluster=rules-db \
  --database-flags=alloydb.iam_authentication=on
```

2. 授予必要 IAM 角色

如要使用受管理 AlloyDB MCP 伺服器，您的身分必須具備特定權限。執行下列指令，授予必要角色：

```
export USER_EMAIL=$(gcloud config get-value account)

# Role to use MCP tools
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="user:$USER_EMAIL" \
  --role="roles/mcp.toolUser"

# Role to execute SQL in AlloyDB
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="user:$USER_EMAIL" \
  --role="roles/alloydb.admin"

# Role for IAM database authentication
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="user:$USER_EMAIL" \
  --role="roles/alloydb.databaseUser"

# Create the IAM-based database user
gcloud alloydb users create "$USER_EMAIL" \
  --cluster=rules-db \
  --region=us-central1 \
  --type=IAM_BASED
```

3. 透過 AlloyDB Studio 建立資料庫和資料表

由於 AlloyDB 資料庫和資料表是透過 SQL 管理，因此我們將使用 Google Cloud 控制台中的 **AlloyDB Studio** 完成結構定義。

1. 依序前往「AlloyDB」>「叢集」，然後點選 `rules-db`。
2. 在左側導覽選單中，按一下「AlloyDB Studio」。
3. 使用 **postgres** 使用者和您設定的密碼 (`postgres`) 登入。
4. 執行下列 SQL 建立資料庫：

```
CREATE DATABASE city_rules;
```

5. 在 AlloyDB Studio 中將資料庫連線切換為 `city_rules`，然後執行下列 SQL 指令，安裝擴充功能並建立 `rules` 資料表：

```
-- Install extensions for vector search and ML
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS google_ml_integration CASCADE;

-- Create the rules table
CREATE TABLE IF NOT EXISTS rules (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    text TEXT NOT NULL,
    city TEXT NOT NULL,
    embedding vector(3072) DEFAULT NULL
);

-- Grant your IAM user access to the table (replace with your email)
GRANT ALL PRIVILEGES ON TABLE rules TO "YOUR_EMAIL_ADDRESS";
```

7. 使用 Spark Lightning Engine 擷取城市規則資料

如要提供真正準確的規劃，代理不只需要精心設計的提示，還需要根據資料和機構背景資訊做出判斷。在這個步驟中，您會使用 Dataproc Serverless 上的 **Spark Lightning Engine** 處理大型城市規則 PDF，並將其擷取至 AlloyDB。

為什麼要使用 Spark Lightning Engine ？

大規模奠定基礎需要處理大量非結構化資料。**Spark Lightning Engine** 是 Spark 的高效能執行引擎，可大幅加速處理這些工作負載。我們在此使用這項技術，透過 Google 的 **Document AI** 對文件執行語意分塊。

探索 Spark 管道

擷取邏輯是在 `spark-setup/spark_alloydb_processor.py` 中定義。管線會依序執行下列步驟：

1. **列出 PDF**：從 Google Cloud Storage 值區擷取文件 URI。
2. **語意擷取**：使用 UDF (使用者定義函式) 呼叫 Document AI API。
3. **寫入 AlloyDB**：將擷取的文字區塊儲存至名為 `rules` 的 AlloyDB 資料表。

```
# Extract from spark_alloydb_processor.py
def process_document(gcs_uri: str):
    # ... calls Document AI to parse PDF ...
    return chunks

# Parallel processing with Spark Lightning Engine
process_udf = udf(process_document, chunk_schema)
chunked_df = uri_df.withColumn("chunks", process_udf(col("gcs_uri"))) \
    .select(explode(col("chunks")).alias("chunk")) \
    .select("chunk.*")

# Save to AlloyDB for Vector Search
chunked_df.write.format("jdbc") \
    .option("url", jdbc_url) \
    .option("dbtable", "rules") \
    .mode("append") \
    .save()
```

執行擷取工作

使用提供的指令碼觸發擷取程序：

```
./spark-setup/run_dataproc.sh
```

透過 Spark Lightning Engine 為代理提供額外資料和背景資訊，確保代理的計畫不僅有創意，也完全符合當地法規。

步驟 8 · 約 10 分鐘

8. 搭配 AlloyDB 使用 RAG

現在城市規則資料已存放在 AlloyDB 中，代理程式可以運用這些資料執行**檢索增強生成 (RAG)**。確保馬拉松計畫符合特定城市代碼。

AlloyDB 為 RAG 帶來的效益

AlloyDB 擅長向量搜尋，因此我們可以在同一處儲存結構化資料和向量嵌入項目。代理程式可以使用 AlloyDB 的內建 `embedding` 函式，找出最相關的規則資訊。

使用 Vector Search 進行混合式擷取

為了讓代理程式存取這項資料，我們提供可使用向量相似度查詢 AlloyDB 的工具。您可以在 `hybrid_recall.sql` 中查看這項邏輯，瞭解如何計算查詢與儲存規則之間的距離：

```
SELECT
  text,
  (embedding <=>
    embedding('gemini-embedding-001',
              'Restrictions for running a race on the Las Vegas strip'))::vector)
  as distance
FROM
  rules
WHERE city = 'Las Vegas'
ORDER BY
  distance ASC
LIMIT 5;
```

使用 RAG 工具，根據當地法規為代理程式建立基準

如要讓代理程式使用這項工具，您必須在 `planner_agent/tools.py` 中定義工具，然後在 `planner_agent/agent.py` 中註冊。我們會使用 Google Cloud 的代管遠端 **AlloyDB MCP** 伺服器連線至資料庫。

1. 使用「混合召回」模式，在 `planner_agent/tools.py` 中定義工具。我們將使用 `streamable_http` 通訊協定連線至受管理 AlloyDB MCP 伺服器：

```

from mcp import ClientSession
from mcp.client.streamable_http import streamablehttp_client

async def get_local_and_traffic_rules(query: str) -> str:
    """Uses vector search in AlloyDB via managed MCP server."""
    # Vector search query using built-in AlloyDB embedding functions
    sql = f"SELECT text FROM rules WHERE city = 'Las Vegas' ORDER BY embedding <=>
google_ml.embedding('gemini-embedding-001', '{query}'):vector ASC LIMIT 5;"

    # Establish a streamable HTTP connection to the MCP server
    async with streamablehttp_client(url, headers=get_auth_headers()) as (read_stream,
write_stream, _):
        async with ClientSession(read_stream, write_stream) as session:
            await session.initialize()
            result = await session.call_tool(
                "execute_sql",
                arguments={
                    "instance": full_instance_name,
                    "database": "city_rules",
                    "sqlStatement": sql
                }
            )
            return "\n".join([c.text for c in result.content if hasattr(c, 'text')])

```

2. 註冊工具並完成 `planner_agent/agent.py` :

```

# ... imports ...

# Assemble the Agent
root_agent = LlmAgent(
    name="planner_agent",
    model="gemini-3-flash-preview",
    instruction="You are a helpful marathon planning assistant...",
    tools=[
        get_local_and_traffic_rules,
    ],
    after_agent_callback=[auto_save_memories],
)

# 2. Wrap the agent in an AdkApp to manage the stateful lifecycle
app = AdkApp(
    agent=root_agent,
    session_service_builder=session_service_builder,
    memory_service_builder=memory_service_builder
)

```

9. 透過 Agent Skills 取得專家指引

Agent Skills 是獨立模組，可提供特定指令、指引和資源，協助代理更有效率地執行工作。不必為每項工具提供複雜的系統提示，您可以將專業知識封裝成技能，只在需要時載入。

Google 提供 **Google 產品** 的預先建構技能 (例如 AlloyDB 和 BigQuery)，確保代理程式遵循業界最佳做法，查詢資料及管理資源。如要瞭解這些和其他專業模式，請前往 [Google Skills Depot](#)。如要瞭解 AlloyDB 的基本技能，請參閱[這篇文章](#)。

1. 探索技能檔案

開啟 `planner_agent/skills/get-local-and-traffic-rules/SKILL.md` 中的預先設定技能檔案。外觀如下：

```
---
name: get-local-and-traffic-rules
description: Retrieve local rules and traffic information for a specific jurisdiction.
---
# get_local_and_traffic_rules Skill

This skill provides guidelines on how to effectively use the `get_local_and_traffic_rules` tool.

## Overview
The `get_local_and_traffic_rules` tool interfaces with an AlloyDB database to perform vector similarity searches on a corpus of rules and traffic information using a provided natural language query.

## Usage Guidelines
1. Query Specificity: When calling the tool, provide specific details in the `query` argument. For example, instead of querying "food rules", use "rules regarding food vendors during public events".
2. Contextual Use: Use the tool when planning events or activities that require adherence to local municipal or state rules (e.g., street closures, noise ordinances, environmental rules).
3. Handling Results: The tool returns a string containing the text of the top 5 most relevant rules. If no error occurs, parse the returned string to inform your planning tasks.
4. Error Handling: If an error string is returned (e.g., "Error querying rules: ..."), you must report this failure or attempt an alternative approach if applicable.

## Underlying Mechanism
- The tool uses `google_ml.embedding` to convert the query into a vector representation.
- It calculates distance (`<=>`) against the `embedding` column in the `rules` table on an AlloyDB instance.
- Results are fetched in descending order of similarity, limited to 5 results.
```

2. 如何註冊技能

在 `planner_agent/agent.py` 中，系統會從目錄載入技能，並新增至代理的工具。程式碼如下所示：


```
import pathlib
from google.adk.skills import load_skill_from_dir
from google.adk.tools import skill_toolset

# Load the AlloyDB skill from its directory
alloydb_skill = load_skill_from_dir(pathlib.Path(__file__).parent / "skills" / "get-local-and-traffic-rules")

# Assemble the Agent with the Skill Toolset
root_agent = LlmAgent(
    name="planner_agent",
    model="gemini-3-flash-preview",
    instruction="You are a helpful marathon planning assistant...",
    tools=[
        get_local_and_traffic_rules,
        skill_toolset.SkillToolset(skills=[alloydb_skill])
    ],
    after_agent_callback=[auto_save_memories],
)
```

代理程式技能可讓您將工具實作與工具指令分開。這樣可確保系統提示詞乾淨，並確保代理程式只在使用特定功能時，才擁有確切的背景資訊。

步驟 10 · 約 5 分鐘

10. 測試代理

1. 在本機啟動代理程式：

```
uv run adk run planner_agent
```

2. 詢問城市規定：`[user]: What are the rules for running a race on the Las Vegas strip?`

代理程式會呼叫 `get_local_and_traffic_rules` 工具、在 AlloyDB 中執行向量搜尋，並根據 Spark 處理的官方規則區塊傳回答案。

步驟 11 · 約 5 分鐘

11. 部署代理程式

部署至 Agent Platform

```
uv run adk deploy agent_engine \  
  --env_file .env \  
  planner_agent
```

12. 清除所用資源

如要避免持續產生費用，請刪除在本程式碼研究室中建立的資源。

刪除 AlloyDB 叢集

```
# Delete the AlloyDB Cluster
gcloud alloydb clusters delete rules-db --region=us-central1 --force
```

刪除 Agent Runtime 應用程式

您可以透過控制台或使用 `gcloud` 指令 (如果您有資源名稱) 刪除 Reasoning Engine 執行個體。為求簡單起見，請使用控制台：

1. 前往「Agent Runtime」頁面。
 2. 選取 `planner_agent` -> 按一下右側的三點按鈕。
 3. 按一下「刪除」。
-

13. 恭喜

恭喜！您已成功使用進階記憶體和資料基礎功能，強化 ADK 代理程式。

目前所學內容

- 有狀態代理：整合 **Agent Platform** 工作階段，維持對話脈絡。
- 長期學習：附加 **Agent Platform Memory Bank**，讓代理程式從使用者互動中學習。
- 資料擷取：使用 **Spark Lightning Engine** 和 **Document AI** 處理非結構化文件。
- **RAG**：在 **AlloyDB** 中建構向量搜尋系統，讓代理程式以現實世界的規則為依據。

後續步驟

- 請參閱 [Agent Platform 說明文件](#)，進一步瞭解代管部署作業。
 - 深入探索 [AlloyDB 向量搜尋](#)，掌握進階 RAG 模式。
 - 使用 [Dataproc Serverless](#) 擴充擷取管道。
-